

Chapitre 7 : Introduction à la programmation orientée objet

Par **Robert DIASSÉ**, Ingénieur Logiciel et Data

> **Cours** : Python avancé · **Fil rouge** : le registre des colis de Yonnee Express · **Version** : 2026-10-31

Fiche du chapitre

Objectifs	A la fin du chapitre, l'apprenant sait : 1) définir une **classe**, créer des **objets**, écrire `__init__` et des méthodes, et expliquer `self` ; 2) distinguer attributs d'instance et attributs de classe ; 3) protéger les données d'un objet (convention `_`, `property`) ; 4) écrire les méthodes spéciales `__str__`, `__repr__`, `__eq__`, `__lt__`, `__len__` ; 5) faire collaborer des objets (**composition**) ; 6) écrire une classe fille (**héritage**, `super()`, redéfinition) et profiter du **polymorphisme** ; 7) écrire une **classe abstraite** avec `abc` ; 8) créer ses propres exceptions en sachant ce qu'est une classe d'exception ; 9) utiliser `@dataclass` pour les classes qui portent surtout des données.
Prérequis	Chapitres 1 à 6 ; votre paquet `yonnee` (atelier 6)
Durée	🟢 Débutant : 10 h · 🔵 Standard : 7 h · 🟣 Expert : 5 h (atelier compris)
Matériel	Python 3.12 ou plus récent (vérifié avec Python 3.13), VS Code, terminal
Livrable	L'atelier 7 : le paquet `yonnee` réécrit avec des classes, avec les mêmes résultats, publié sur GitHub

Comment lire ce chapitre

- **Noyau** : obligatoire pour tous les niveaux.
- 🟢 **Débutant** : explications pas à pas.
- 🔵 **Standard** : le niveau de référence.
- 🟣 **Expert** : approfondissements.

Créez un dossier `essais-ch7`, un fichier par section (`s1_premiere_classe.py`, `s2_...`). Les exemples parlent des **colis** de Yonnee Express ; l'atelier, lui, réécrira le **registre** des livraisons.

D'où l'on part

Le chapitre 6 (section 8) a montré trois limites du paquet `yonnee` : une `Livraison` (un tuple nommé) peut être créée avec des valeurs fausses, car la vérification est dans **une autre** fonction ; pour savoir si une livraison est en retard, il faut savoir **quelle** fonction appeler et **où** ; et un tuple nommé n'a aucun comportement à lui. La **programmation orientée objet** (POO) réunit, dans une même **classe**, les **données** et les **opérations** qui les concernent.

> 🟢 **Débutant : vous utilisez des objets depuis toujours** En Python, **tout** est objet : `"Dakar".upper()` appelle la méthode `upper` de l'objet texte `"Dakar"` ; `livraisons.append(1)` appelle la méthode `append` d'un objet liste. `type("Dakar")` répond `<class 'str'>` : `str` est une classe, et `"Dakar"` un objet de cette classe. Ce chapitre vous apprend à écrire **vos propres** classes.

---

1. Classe, objet, `__init__` et `self`

```
class Colis:
    def __init__(self, destinataire, quartier, poids):
        self.destinataire = destinataire
        self.quartier = quartier
        self.poids = poids

    def est_lourd(self):
        return self.poids > 5

    def decrire(self):
        return f"Colis pour {self.destinataire} ({self.quartier}, {self.poids} kg)"


c1 = Colis("Awa Diop", "Pikine", 7.5)
c2 = Colis("Moussa Sarr", "Médina", 1.2)
print(c1.decrire())
print(c2.decrire())
print(c1.est_lourd(), c2.est_lourd())
c2.poids = 6
print(c2.decrire(), c2.est_lourd())
print(type(c1))
print(c1)
```

```
Colis pour Awa Diop (Pikine, 7.5 kg)
Colis pour Moussa Sarr (Médina, 1.2 kg)
True False
Colis pour Moussa Sarr (Médina, 6 kg) True
<class '__main__.Colis'>
<__main__.Colis object at 0x0000012647946900>
```

(L'adresse `0x...` de la dernière ligne change à chaque exécution.)

Morceau	Ce qu'il veut dire
`class Colis:`	Définit une **classe** : un modèle de colis. Par convention, son nom commence par une majuscule (`Colis` , `Tournee`)
`def __init__(self, destinataire, quartier, poids):`	Le **constructeur** : Python l'appelle **automatiquement** à chaque création d'objet. Les deux soulignés de chaque côté marquent une méthode spéciale
`self`	L'objet **en train d'être construit**, ou sur lequel on appelle la méthode. Python le passe tout seul : on écrit `c1.est_lourd()` , Python exécute `Colis.est_lourd(c1)`
`self.poids = poids`	Crée un **attribut** de l'objet : une variable rangée **dans** l'objet. A gauche l'attribut, à droite le paramètre
`def est_lourd(self):`	Une **méthode** : une fonction rangée dans la classe, qui travaille sur **son** objet. Son premier paramètre est toujours `self`
`c1 = Colis("Awa Diop", "Pikine", 7.5)`	Crée un **objet** (une **instance**) : Python crée un objet vide, appelle `__init__` , et renvoie l'objet rempli
`c2.poids = 6`	Modifie l'attribut de `c2` seulement ; `c1` n'est pas touché
`print(c1)`	Sans méthode spéciale `__str__` (section 4), Python affiche la classe et une adresse en mémoire

🖍️ **Essayez** : retirez `self` de `def est_lourd(self):` . Message : `TypeError: Colis.est_lourd() takes 0 positional arguments but 1 was given` Traduction : « la méthode ne prend aucun argument, mais un lui a été donné ». Python a passé l'objet (`self`) tout seul, et la méthode n'avait pas de place pour lui. Annulez.

🖍️ **Essayez** : `print(c1.poid)` (sans le *s*). Message : `AttributeError: 'Colis' object has no attribute 'poid'. Did you mean: 'poids'?` . Puis créez `Colis("Awa Diop", "Pikine")` (un argument de moins) : `TypeError: Colis.__init__() missing 1 required positional argument: 'poids'` . Annulez.

🖍️ **Essayez** : dans `__init__` , écrivez `destinataire = destinataire` (sans `self.`). Aucune erreur à la création, mais `c1.decrire()` donne `AttributeError: 'Colis' object has no attribute 'destinataire'` : sans `self.` , on a créé une variable **locale**, oubliée à la fin de `__init__` . Annulez.

2. Attributs d'instance et attributs de classe

```
class Colis:
    nombre_crees = 0
    POIDS_LOURD = 5

    def __init__(self, destinataire, quartier, poids):
        Colis.nombre_crees += 1
        self.numero = Colis.nombre_crees
        self.destinataire = destinataire
        self.quartier = quartier
        self.poids = poids

    def est_lourd(self):
        return self.poids > self.POIDS_LOURD


a = Colis("Awa Diop", "Pikine", 7.5)
b = Colis("Moussa Sarr", "Médina", 1.2)
c = Colis("Fatou Ba", "Yoff", 3.0)
print(a.numero, b.numero, c.numero)
print("Colis créés :", Colis.nombre_crees)
print(a.est_lourd(), Colis.POIDS_LOURD)
```

```
1 2 3
Colis créés : 3
True 5
```

Morceau	Ce qu'il veut dire
`nombre_crees = 0` écrit dans la classe, hors des méthodes	Un **attribut de classe** : il existe **une seule fois**, partagé par tous les colis
`Colis.nombre_crees += 1`	On le modifie **par le nom de la classe**
`self.numero = ...`	Un **attribut d'instance** : chaque colis a le sien
`POIDS_LOURD = 5` et `self.POIDS_LOURD`	Une constante de classe ; on peut la lire par l'objet (`self.`) ou par la classe

> ⚠️ **Le piège de la liste de classe.** Une **liste** (ou un dictionnaire) écrite au niveau de la classe est **partagée par tous les objets** : si chaque groupe doit avoir ses membres, créez la liste **dans** `__init__` (`self.membres = []`). L'exercice 2 vous fait prédire ce piège.

3. Protéger les données : `_` et `property`

Rien n'empêche d'écrire `c.poids = -2` : le colis devient faux. Python n'a pas de `private` comme Java, mais deux outils :

- un **soulignement** devant un nom (`self._poids`) dit « interne, ne pas toucher de l'extérieur » : c'est une **convention**, que tous les programmeurs Python respectent ;
- une **propriété** (`@property`) : on accède à `c.poids` comme à un attribut, mais Python appelle des **méthodes**, qui peuvent vérifier.

```
class Colis:
    def __init__(self, destinataire, quartier, poids):
        self.destinataire = destinataire
        self.quartier = quartier
        self.poids = poids

    @property
    def poids(self):
        return self._poids

    @poids.setter
    def poids(self, valeur):
        if valeur <= 0:
            raise ValueError(f"poids invalide : {valeur}")
        self._poids = valeur

    @property
    def est_lourd(self):
        return self.poids > 5


c = Colis("Awa Diop", "Pikine", 7.5)
print(c.poids, c.est_lourd)
c.poids = 3.2
print(c.poids, c.est_lourd)
try:
    c.poids = -2
except ValueError as e:
    print("Refusé :", e)
print(c.poids)
try:
    Colis("Moussa Sarr", "Médina", 0)
except ValueError as e:
    print("Refusé :", e)
```

```
7.5 True
3.2 False
Refusé : poids invalide : -2
3.2
Refusé : poids invalide : 0
```

Morceau	Ce qu'il veut dire
`@property` sur `def poids(self)`	Lire `c.poids` appelle cette méthode, qui renvoie la vraie valeur, rangée dans `self._poids`
`@poids.setter`	Écrire `c.poids = ...` appelle cette méthode, qui **vérifie** avant de ranger

Morceau	Ce qu'il veut dire
`self.poids = poids` dans `__init__`	Passe **aussi** par le *setter* : un colis ne peut pas **naître** avec un poids faux
`@property def est_lourd`	Une valeur **calculée**, qui se lit comme un attribut : `c.est_lourd`, sans parenthèses

> 🧨 **Essayez** : `c.est_lourd = True`. Message : `AttributeError: property 'est_lourd' of 'Colis' object has no setter`. Une propriété sans *setter* est en **lecture seule** : on ne peut pas décréter qu'un colis est lourd. Annulez.

4. Les méthodes spéciales

Les méthodes entourées de doubles soulignés (on dit *dunder*, pour *double underscore*) permettent à vos objets de fonctionner avec `print`, `==`, `<`, `sorted`, `len`…

```
class Colis:
    def __init__(self, destinataire, quartier, poids):
        self.destinataire = destinataire
        self.quartier = quartier
        self.poids = poids

    def __str__(self):
        return f"{self.destinataire} ({self.quartier}, {self.poids} kg)"

    def __repr__(self):
        return f"Colis({self.destinataire!r}, {self.quartier!r}, {self.poids!r})"

    def __eq__(self, autre):
        if not isinstance(autre, Colis):
            return NotImplemented
        return (self.destinataire, self.quartier, self.poids) == (autre.destinataire,
         autre.quartier, autre.poids)

    def __lt__(self, autre):
        return self.poids < autre.poids


a = Colis("Awa Diop", "Pikine", 7.5)
b = Colis("Moussa Sarr", "Médina", 1.2)
c = Colis("Awa Diop", "Pikine", 7.5)
print(a)
print(repr(a))
print([a, b])
print(a == c, a is c, a == b)
print(sorted([a, b, Colis("Fatou Ba", "Yoff", 3.0)]))
print(max([a, b]))
```

```
Awa Diop (Pikine, 7.5 kg)
Colis('Awa Diop', 'Pikine', 7.5)
[Colis('Awa Diop', 'Pikine', 7.5), Colis('Moussa Sarr', 'Médina', 1.2)]
True False False
[Colis('Moussa Sarr', 'Médina', 1.2), Colis('Fatou Ba', 'Yoff', 3.0), Colis('Awa Diop', 'Pikine',
7.5)]
Awa Diop (Pikine, 7.5 kg)
```

Méthode	Appelée par	Rôle
`__str__`	`print(objet)`, `str(objet)`, les *f-strings*	Une description **pour l'utilisateur**
`__repr__`	`repr(objet)`, l'affichage d'une **liste** d'objets, la console	Une description **pour le programmeur**, si possible du code Python qui recrée l'objet (`!r` ajoute les guillemets)
`__eq__`	`a == b`	Quand deux objets sont-ils « égaux » ? Sans elle, `==` compare les identités, comme `is`
`__lt__`	`a < b`, `sorted`, `min`, `max`	L'ordre (*less than*) : ici, par poids
`__len__`	`len(objet)`	La « taille » de l'objet (section 5)

Morceau	Ce qu'il veut dire
`a == c` vaut `True`, `a is c` vaut `False`	Deux colis **distincts** (`is` : est-ce le même objet ?), mais **égaux** selon notre règle
`return NotImplemented`	« Je ne sais pas comparer avec cet objet-là » : Python répond alors `False` à `a == "texte"`, au lieu de planter

> 🪛 **Essayez** : supprimez `__lt__`, puis relancez. Message : `TypeError: '<' not supported between instances of 'Colis' and 'Colis'`. Sans ordre défini, Python ne peut pas trier des colis. Annulez.

5. La composition : des objets qui contiennent des objets

Une tournée de livreur **contient** des colis : un attribut de la classe `Tournee` est une **liste d'objets** `Colis`.

```python
class Colis:
    def __init__(self, destinataire, quartier, poids):
        self.destinataire = destinataire
        self.quartier = quartier
        self.poids = poids

    def __str__(self):
        return f"{self.destinataire} ({self.quartier}, {self.poids} kg)"


class Tournee:
    def __init__(self, livreur, charge_max):
        self.livreur = livreur
        self.charge_max = charge_max
        self.colis = []

    def charge(self):
        return sum(c.poids for c in self.colis)

    def ajouter(self, colis):
        if self.charge() + colis.poids > self.charge_max:
            return False
        self.colis.append(colis)
        return True

    def __len__(self):
        return len(self.colis)

    def __str__(self):
        return f"Tournée de {self.livreur} : {len(self)} colis, {self.charge()} kg sur
         {self.charge_max}"


t = Tournee("Ibou", 20)
for c in [Colis("Awa Diop", "Pikine", 7.5), Colis("Moussa Sarr", "Médina", 1.2),
          Colis("Fatou Ba", "Yoff", 9.0), Colis("Khady Ndiaye", "Pikine", 4.0)]:
    print(f"{c} : {'accepté' if t.ajouter(c) else 'refusé'}")
print(t)
for c in t.colis:
    print(" -", c)
```

```
Awa Diop (Pikine, 7.5 kg) : accepté
Moussa Sarr (Médina, 1.2 kg) : accepté
Fatou Ba (Yoff, 9.0 kg) : accepté
Khady Ndiaye (Pikine, 4.0 kg) : refusé
Tournée de Ibou : 3 colis, 17.7 kg sur 20
 - Awa Diop (Pikine, 7.5 kg)
 - Moussa Sarr (Médina, 1.2 kg)
 - Fatou Ba (Yoff, 9.0 kg)
```

Morceau	Ce qu'il veut dire
`self.colis = []`	Chaque tournée a **sa propre** liste, créée dans `__init__` (le piège de la section 2 évité)
`sum(c.poids for c in self.colis)`	La tournée **demande** son poids à chaque colis : 7,5 + 1,2 + 9,0 = 17,7 kg
`ajouter` refuse Khady	17,7 + 4,0 = 21,7 kg dépasserait les 20 kg autorisés
`def __len__` et `len(self)`	`len(t)` fonctionne sur une tournée, comme sur une liste

On parle de **composition** : la tournée **a des** colis. C'est la relation la plus fréquente entre classes.

6. L'héritage et le polymorphisme

6.1 Une classe fille

Certains colis coûtent plus cher : un colis **fragile** a un supplément de 1000 F, un colis **express** coûte le double. Ce sont des colis, avec une règle de frais différente : on écrit des classes **filles**, qui **héritent** de `Colis`.

```python
class Colis:
    TARIFS = {"Médina": 1000, "Plateau": 1500, "Yoff": 2000, "Parcelles": 2000, "Pikine": 2500}

    def __init__(self, destinataire, quartier, poids):
        self.destinataire = destinataire
        self.quartier = quartier
        self.poids = poids

    def frais(self):
        frais = self.TARIFS[self.quartier]
        if self.poids > 5:
            frais += 500
        return frais

    def __str__(self):
        return f"{self.destinataire} ({self.quartier}, {self.poids} kg) : {self.frais()} F"


class ColisFragile(Colis):
    SUPPLEMENT = 1000

    def __init__(self, destinataire, quartier, poids, contenu):
        super().__init__(destinataire, quartier, poids)
        self.contenu = contenu

    def frais(self):
        return super().frais() + self.SUPPLEMENT

    def __str__(self):
        return super().__str__() + f", fragile : {self.contenu}"


class ColisExpress(Colis):
    def frais(self):
        return super().frais() * 2


colis = [
    Colis("Awa Diop", "Pikine", 7.5),
    ColisFragile("Moussa Sarr", "Médina", 1.2, "vaisselle"),
    ColisExpress("Fatou Ba", "Yoff", 3.0),
]
for c in colis:
    print(c)
print("Total :", sum(c.frais() for c in colis), "F")
print(isinstance(colis[1], Colis), isinstance(colis[0], ColisFragile))
print(ColisFragile.__mro__)
```

```
Awa Diop (Pikine, 7.5 kg) : 3000 F
Moussa Sarr (Médina, 1.2 kg) : 2000 F, fragile : vaisselle
Fatou Ba (Yoff, 3.0 kg) : 4000 F
Total : 9000 F
True False
(<class '__main__.ColisFragile'>, <class '__main__.Colis'>, <class 'object'>)
```

Morceau	Ce qu'il veut dire
`class ColisFragile(Colis):`	`ColisFragile` **hérite** de `Colis` : elle reçoit tous ses attributs et méthodes
`super().__init__(destinataire, quartier, poids)`	Appelle le constructeur de la **mère**, qui range les trois attributs communs ; la fille range ensuite **le sien** (`contenu`)
`def frais(self): return super().frais() + ...`	**Redéfinit** la méthode de la mère, en réutilisant son calcul (`super().frais()`)
`ColisExpress` sans `__init__`	Pas besoin : elle **hérite** du constructeur de la mère
`ColisExpress` : 4000 F	Tarif de Yoff (2000), pas de supplément (3 kg), doublé
`isinstance(colis[1], Colis)` vaut `True`	Un colis fragile **est un** colis
`__mro__`	L'ordre dans lequel Python cherche une méthode : la fille, puis la mère, puis `object` , la mère de toutes les classes

6.2 Le polymorphisme

Regardez `sum(c.frais() for c in colis)` : la **même** ligne appelle **trois** règles de frais différentes, selon la vraie classe de chaque objet. C'est le **polymorphisme** : le code qui utilise les colis n'a pas besoin de savoir quelle sorte de colis il manipule. Ajouter demain un `ColisFrigorifique` ne demandera **aucune** modification de cette ligne.

> 🖍️ **Essayez** : dans `ColisFragile.__init__` , remplacez la ligne `super().__init__(...)` par `self.poids = poids` . Message : `AttributeError: 'ColisFragile' object has no attribute 'destinataire'` . Sans `super().__init__` , la partie « colis » n'est pas construite. Annulez.

> 🔵 **Standard : le « duck typing »** En Python, le polymorphisme ne demande même pas d'héritage : tout objet qui a une méthode `frais()` peut aller dans la liste. « Si ça marche comme un canard et que ça cancane comme un canard, c'est un canard. » L'héritage reste utile pour **partager** du code et pour **dire** clairement qu'une classe est une sorte d'une autre.

7. Les classes abstraites

Un « véhicule » en général ne veut rien dire pour Yonnee Express : il y a des motos et des fourgonnettes. On veut une classe mère qui **impose** à chaque véhicule de dire sa charge maximale et son coût au kilomètre, sans pouvoir créer de « véhicule » tout court. C'est une **classe abstraite**, du module `abc` (*Abstract Base Classes*).

```
from abc import ABC, abstractmethod


class Vehicule(ABC):
    def __init__(self, immatriculation):
        self.immatriculation = immatriculation

    @abstractmethod
    def charge_max(self):
        """La charge maximale en kg."""

    @abstractmethod
    def cout_km(self):
        """Le coût en carburant par km, en FCFA."""

    def cout_tournee(self, km):
        return round(self.cout_km() * km)

    def __str__(self):
        return f"{type(self).__name__} {self.immatriculation} : {self.charge_max()} kg max"


class Moto(Vehicule):
    def charge_max(self):
        return 30

    def cout_km(self):
        return 45


class Fourgonnette(Vehicule):
    def charge_max(self):
        return 800

    def cout_km(self):
        return 120


flotte = [Moto("DK-2041-A"), Fourgonnette("DK-7310-B"), Moto("DK-9902-C")]
for v in flotte:
    print(v, "| 35 km :", v.cout_tournee(35), "F")
try:
    Vehicule("DK-0000-Z")
except TypeError as e:
    print("Refusé :", e)
```

```
Moto DK-2041-A : 30 kg max | 35 km : 1575 F
Fourgonnette DK-7310-B : 800 kg max | 35 km : 4200 F
Moto DK-9902-C : 30 kg max | 35 km : 1575 F
Refusé : Can't instantiate abstract class Vehicule without an implementation for abstract methods
'charge_max', 'cout_km'
```

Morceau	Ce qu'il veut dire
`class Vehicule(ABC):`	Une classe **abstraite** : un modèle incomplet
`@abstractmethod`	Une méthode que **chaque fille doit écrire**. Son corps est vide (ici, seulement une *docstring*)
`cout_tournee`	Une méthode **ordinaire** de la mère, qui utilise les méthodes abstraites : 45 × 35 = 1575 F pour une moto
`type(self).__name__`	Le nom de la vraie classe de l'objet (`Moto` , `Fourgonnette`)
`Vehicule(...)` refusé	Traduction : « impossible de créer un objet de la classe abstraite `Vehicule` sans les méthodes abstraites `charge_max` et `cout_km` »

> 🖍️ **Essayez** : retirez la méthode `cout_km` de `Moto` . Message, à la création d'une moto : `TypeError: Can't instantiate abstract class Moto without an implementation for abstract method 'cout_km'` . L'oubli est détecté **dès la création** de l'objet, et non au milieu d'une tournée. Annulez.

8. Les exceptions personnalisées, enfin expliquées

Au chapitre 1 (encadré de la section sur les exceptions personnalisées), vous avez écrit `class MontantInvalideError(ValueError):` sans bien savoir ce que c'était. Vous pouvez maintenant le lire : une exception est une **classe**, qui **hérite** d' `Exception` ; `raise` crée un **objet** de cette classe et le lance ; `except` l'attrape **par sa classe, ou par une de ses mères**.

```python
class ErreurYonnee(Exception):
    """Mère de toutes les erreurs de l'application."""


class ColisInvalide(ErreurYonnee, ValueError):
    def __init__(self, champ, valeur):
        super().__init__(f"{champ} invalide : {valeur!r}")
        self.champ = champ
        self.valeur = valeur


class SurchargeTournee(ErreurYonnee):
    def __init__(self, charge, maximum):
        super().__init__(f"surcharge : {charge} kg pour {maximum} kg autorisés")
        self.depassement = charge - maximum


def verifier(poids, quartier):
    if poids <= 0:
        raise ColisInvalide("poids", poids)
    if quartier not in ("Médina", "Plateau", "Yoff", "Parcelles", "Pikine"):
        raise ColisInvalide("quartier", quartier)


for poids, quartier in [(2.0, "Yoff"), (-1, "Yoff"), (3.0, "Thiès")]:
    try:
        verifier(poids, quartier)
        print("ok :", poids, quartier)
    except ColisInvalide as e:
        print("Rejeté :", e, "| champ :", e.champ)

try:
    raise SurchargeTournee(23.5, 20)
except ErreurYonnee as e:
    print(type(e).__name__, ":", e, "| dépassement :", e.depassement)

try:
    verifier(0, "Yoff")
except ValueError as e:
    print("Attrapée comme ValueError :", e)
print(ColisInvalide.__mro__)
```

```
ok : 2.0 Yoff
Rejeté : poids invalide : -1 | champ : poids
Rejeté : quartier invalide : 'Thiès' | champ : quartier
SurchargeTournee : surcharge : 23.5 kg pour 20 kg autorisés | dépassement : 3.5
Attrapée comme ValueError : poids invalide : 0
(<class '__main__.ColisInvalide'>, <class '__main__.ErreurYonnee'>, <class 'ValueError'>, <class
'Exception'>, <class 'BaseException'>, <class 'object'>)
```

Morceau	Ce qu'il veut dire
`class ErreurYonnee(Exception)`	Une exception **mère** de l'application : un seul `except ErreurYonnee` attrape toutes ses erreurs
`class ColisInvalide(ErreurYonnee, ValueError)`	Deux mères (l'**héritage multiple**, permis en Python) : c'est une erreur Yonnee **et** une `ValueError`, donc le code existant qui attrape `ValueError` continue de marcher
`super().__init__(message)`	Le message que `print(e)` affichera
`self.champ`, `self.depassement`	Une exception peut porter des informations utiles à celui qui l'attrape

9. `@dataclass` : les classes qui portent surtout des données

Écrire `__init__`, `__repr__` et `__eq__` pour une classe qui range surtout des données est répétitif. Le décorateur `@dataclass` (module `dataclasses`, dans la bibliothèque standard) les écrit pour vous, à partir de **annotations** de types.

```
from dataclasses import dataclass, field


@dataclass
class Adresse:
    quartier: str
    rue: str
    repere: str = ""


@dataclass(order=True)
class Colis:
    poids: float
    destinataire: str
    adresse: Adresse
    options: list = field(default_factory=list)

    def __post_init__(self):
        if self.poids <= 0:
            raise ValueError(f"poids invalide : {self.poids}")


a = Colis(7.5, "Awa Diop", Adresse("Pikine", "Rue 10", "près de la mosquée"))
b = Colis(1.2, "Moussa Sarr", Adresse("Médina", "Rue 6"), ["fragile"])
print(a)
print(b.adresse.quartier, b.options)
print(a == Colis(7.5, "Awa Diop", Adresse("Pikine", "Rue 10", "près de la mosquée")))
print(sorted([a, b])[0].destinataire)
try:
    Colis(0, "Fatou Ba", Adresse("Yoff", "Rue 2"))
except ValueError as e:
    print("Refusé :", e)
```

```
Colis(poids=7.5, destinataire='Awa Diop', adresse=Adresse(quartier='Pikine', rue='Rue 10',
repere='près de la mosquée'), options=[])
Médina ['fragile']
True
Moussa Sarr
Refusé : poids invalide : 0
```

Morceau	Ce qu'il veut dire
`@dataclass`	Écrit tout seul `__init__`, `__repr__` et `__eq__` à partir des champs annotés
`quartier: str`	Un **champ** et son type. Le type est une **indication** (Python ne le vérifie pas à l'exécution)
`repere: str = ""`	Une valeur par défaut ; les champs avec défaut viennent **après** ceux sans
`@dataclass(order=True)`	Écrit aussi `__lt__`, `__le__`… : les colis se comparent champ par champ, **dans l'ordre** des champs (d'abord le poids)
`field(default_factory=list)`	Une **nouvelle** liste vide pour chaque colis (jamais `options: list = []`)
`__post_init__`	Appelée juste après l'`__init__` écrit par `dataclass` : l'endroit où **vérifier**
`adresse: Adresse`	La composition, encore : un colis **a une** adresse

> 🧨 **Essayez** : écrivez `options: list = []`. Message, dès la définition de la classe : `ValueError: mutable default <class 'list'> for field options is not allowed: use default_factory`. Python vous protège du piège de la section 2. Annulez.

> 🔵 **Standard : classe ordinaire ou `dataclass` ?** `dataclass` convient aux objets qui **portent des données** et ont peu de comportement (une adresse, une livraison du registre). Une classe ordinaire convient mieux quand la logique domine (une tournée, un véhicule), ou quand il faut des propriétés avec *setter*.

10. Méthode de classe

Une méthode marquée `@classmethod` reçoit la **classe** (`cls`) au lieu d'un objet : on s'en sert surtout comme **autre façon de construire** un objet, par exemple depuis une ligne de fichier :

```python
@classmethod
def depuis_ligne(cls, ligne, tarifs):
    ...
    return cls(numero, jour, livreur, quartier, poids, montant, frais, duree)
```

On l'appelle sur la classe : `Livraison.depuis_ligne(ligne, tarifs)`. Vous en écrirez une à l'atelier 7.

Exercices

Créez un dossier `exercices-ch7`.

- ★ **Exercice 1 : la carte de transport.** Classe `CarteTransport` : un titulaire, un solde **en lecture seule** (`property` sans *setter*), un nombre de trajets ; `recharger(montant)` (refuse un montant nul ou négatif avec `ValueError`) ; `payer_trajet()` (200 F, attribut de classe ; lève une exception `SoldeInsuffisant` si le solde ne suffit pas) ; `__str__`. Test : carte de Ndeye avec 500 F, deux trajets, un troisième refusé (100 F restants), recharge de 1000 F, un trajet : `Carte de Ndeye : 900 F, 3 trajets`. Essayez `carte.solde = 99999`. (Sections 1 à 4.)
- ★ **Exercice 2 : prédire avant d'exécuter.** Écrivez sur une feuille ce qu'affichent les trois `print`, **puis** vérifiez. Expliquez chaque résultat avec les mots « attribut de classe » et « attribut d'instance ». (Section 2.)

```
class Groupe:
    membres = []
    total = 0

    def __init__(self, nom):
        self.nom = nom
        self.total = 0

    def ajouter(self, personne):
        self.membres.append(personne)
        self.total += 1
        Groupe.total += 1


a = Groupe("A")
b = Groupe("B")
a.ajouter("Awa")
b.ajouter("Bineta")
b.ajouter("Babacar")
print(a.total, b.total, Groupe.total)
print(a.membres)
print(a.membres is b.membres)
```

- ★★ **Exercice 3 : les forfaits téléphoniques.** Classe abstraite `Forfait` (un numéro) avec la méthode abstraite `cout(minutes, mo)` (le coût d'un mois). Trois filles : `SansForfait` (50 F la minute, 10 F le Mo), `ForfaitAppels` (5000 F, 300 minutes comprises puis 40 F la minute, 10 F le Mo), `ForfaitInternet` (7000 F, 10 000 Mo compris puis 2 F le Mo, 50 F la minute). Écrivez `moins_cher(forfaits, minutes, mo)` avec `min(..., key=...)`. Résultats attendus : pour 120 min et 500 Mo, le forfait appels (10000 F) ; pour 400 min et 300 Mo, le forfait appels (12000 F) ; pour 60 min et 12 000 Mo, le forfait internet (14000 F). (Sections 6 et 7.)
- ★★ **Exercice 4 : le classement du tournoi.** Un `@dataclass` `Equipe` (nom, points, buts pour, buts contre, liste des matchs) avec une propriété `difference` et un `__lt__` qui compare les points, puis la différence de buts, puis les buts marqués. Une fonction `jouer(e1, buts1, e2, buts2)` met à jour les deux équipes (3 points la victoire, 1 le nul). Jouez : Jaraaf 2-1 Niayes, Thiaroye 0-0 Medina, Jaraaf 1-1

Thiaroye, Niayes 3-0 Medina, Jaraaf 0-2 Medina, Niayes 2-2 Thiaroye, et affichez le classement : Niayes, Jaraaf, Medina (à égalité de points avec Jaraaf, départagées par la différence puis les buts), Thiaroye. (Sections 4 et 9.)

- ★★★ **Exercice 5 : la pharmacie de garde.** Un `@dataclass` `Medicament` (nom, prix, sur ordonnance ou non) ; une classe `Stock` avec `ajouter(medicament, quantite)` (additionne si le médicament existe déjà), `vendre(nom, quantite, avec_ordonnance=False)` (renvoie le montant), `__contains__`, `__len__`, et `alertes()` (les médicaments sous 5 unités, triés). Des exceptions : une mère `ErreurPharmacie`, `MedicamentInconnu` (qui hérite aussi de `KeyError`), `RuptureDeStock` (avec un attribut `manque`), et une `ErreurPharmacie` pour la vente sans ordonnance. Test : paracétamol (500 F) 30 puis 10, amoxicilline (2500 F, ordonnance) 6, sérum oral (300 F) 4 ; ventes : 3 paracétamols, 2 amoxicillines sans puis avec ordonnance, 10 sérums, 1 vitamine C. Caisse : 6500 F ; alertes : amoxicilline et sérum oral. (Toutes les sections.)

Atelier 7 : le registre en classes

Le travail principal du chapitre : vous réécrivez le paquet `yonnee` avec des classes (`Tarifs`, une `Livraison` qui se vérifie elle-même, un `Registre` qui se charge, se filtre et produit ses rapports, une exception `LivraisonInvalide`), et vous retrouvez **exactement** les résultats de l'atelier 6. L'énoncé vous est fourni séparément.

A retenir

- `class Nom:` ; `__init__(self, ...)` construit l'objet ; `self.attribut = valeur` ; une méthode a toujours `self` en premier paramètre ; `objet = Nom(...)`.
- Attribut **de classe** (écrit dans la classe, partagé) contre attribut **d'instance** (`self.x`, propre à chaque objet) ; jamais de liste modifiable au niveau de la classe.
- `_nom` : interne par convention ; `@property` (et `@x.setter`) pour vérifier ou calculer.
- `__str__` (pour l'utilisateur), `__repr__` (pour le programmeur), `__eq__` (`==`), `__lt__` (`<`, `sorted`), `__len__` (`len`).
- **Composition** : un objet a des objets (une tournée a des colis). **Héritage** : `class Fille(Mere)`, `super().__init__(...)`, redéfinir en réutilisant `super().methode()`. **Polymorphisme** : le même appel, des comportements différents.
- Classe abstraite : `class X(ABC)` et `@abstractmethod` ; on ne peut pas en créer d'objet, et une fille qui oublie une méthode abstraite est refusée.
- Une exception est une classe qui hérite d'`Exception` ; une mère commune pour l'application ; des attributs utiles.
- `@dataclass` : `__init__`, `__repr__`, `__eq__` écrits pour vous ; `order=True` ; `field(default_factory=list)` ; `__post_init__` pour vérifier. `@classmethod` pour d'autres constructeurs.

Erreurs fréquentes

Message ou symptôme	Ce que ça veut dire	Solution
`TypeError: X.m() takes 0 positional arguments but 1 was given`	`self` oublié dans la définition de la méthode	`def m(self):`
`AttributeError: 'X' object has no attribute 'y'`	Attribut mal orthographié, ou jamais créé (`self.` oublié dans `__init__`)	Vérifier le nom ; `self.y = ...` dans `__init__`
`TypeError: X.__init__() missing 1 required positional argument`	Pas assez d'arguments à la création	Donner tous les arguments, ou une valeur par défaut
`<__main__.X object at 0x...>` à l'affichage	Pas de `__str__`	Écrire `__str__` (et `__repr__`)
`TypeError: '<' not supported between instances of 'X' and 'X'`	Tri ou `min` / `max` sans ordre défini	`__lt__`, ou `sorted(..., key=...)`, ou `@dataclass(order=True)`
`AttributeError: property 'x' of 'X' object has no setter`	Écriture dans une propriété en lecture seule	Ajouter un `@x.setter`, ou ne pas modifier
Tous les objets partagent la même liste	Liste écrite au niveau de la classe	La créer dans `__init__`
`AttributeError` sur un attribut de la mère, dans une fille	`super().__init__(...)` oublié	L'appeler en première ligne du `__init__` de la fille
`TypeError: Can't instantiate abstract class ...`	Objet d'une classe abstraite, ou fille incomplète	Créer une fille concrète ; écrire toutes les méthodes abstraites
`ValueError: mutable default <class 'list'> for field ...`	Liste par défaut dans une `dataclass`	`field(default_factory=list)`

Pour aller plus loin

- Tutoriel officiel de Python en français, *Classes* : https://docs.python.org/fr/3/tutorial/classes.html
- `dataclasses` : https://docs.python.org/fr/3/library/dataclasses.html ; `abc` : https://docs.python.org/fr/3/library/abc.html
- Les méthodes spéciales (*Data model*) : https://docs.python.org/fr/3/reference/datamodel.html
- La suite logique de ce cours : le typage (`mypy`), les tests (`pytest`) et l'outillage professionnel, prévus par le syllabus après la POO.